GRADO EN INGENIERÍA INFORMÁTICA

CURSO 2024-2025



UNIVERSIDAD
DE GRANADA

SISTEMAS GRÁFICOS

PRÁCTICA 2: CREACIÓN DE UN SISTEMA GRÁFICO LIBRE

CHESS WAR

Francisco Victoriano Pérez Morales

Ángel Rodríguez Faya

26/05/2025

ÍNDICE

1. Estructura del proyecto. 4
2. Comandos para compilar y ejecutar el juego. 4
3. Documentación entregada junto al código. 7
4. Algoritmos principales 7
4.1. Lógica 7
4.2. Modelos 9
4.2. Picking 10

Chess War es un pequeño juego basado en Battle Chess. Se trata de un juego de ajedrez donde las piezas cobran vida y luchan entre ellas, que hemos desarrollado utilizando Three.js y Javascript.

El objetivo es sintetizar los conocimientos adquiridos en la asignatura de Sistemas Gráficos, del Grado en Ingeniería Informática en la Universidad de Granada.

Disponemos de libertad para desarrollar e implementar el juego como deseemos; no obstante, debemos de cumplir unos requisitos mínimos



1. Estructura del proyecto.

Nuestro proyecto tendría la siguiente estructura (de directorios y ficheros más importantes)

practica_2_chess_war

- codigo/
  - chess_war/
    - animacion/
      - index.html
      - MyScene.js
    - game/
      - logica/
      - modelos/
      - index.html
      - MyScene.js
    - imgs/
    - libs/
    - models/
    - probar_piezas/
      - index.html
      - MyScene.js
    - utils/
    - server_launcher.py
  - mostrar_escena_lucha_en_accion/
    - Readme.txt
  - mostrar_piezas_por_separado/
    - Readme.txt
- documentacion/
  - concrecion_plantilla_chess_war.pdf
  - Diagrama_clases_chess_war.png
  - Modelo_jerarquico_peon.png
  - video_animacion.txt

2. Comandos para compilar y ejecutar el juego.

Nuestro proyecto tendría la siguiente estructura (de directorios y ficheros más importantes)

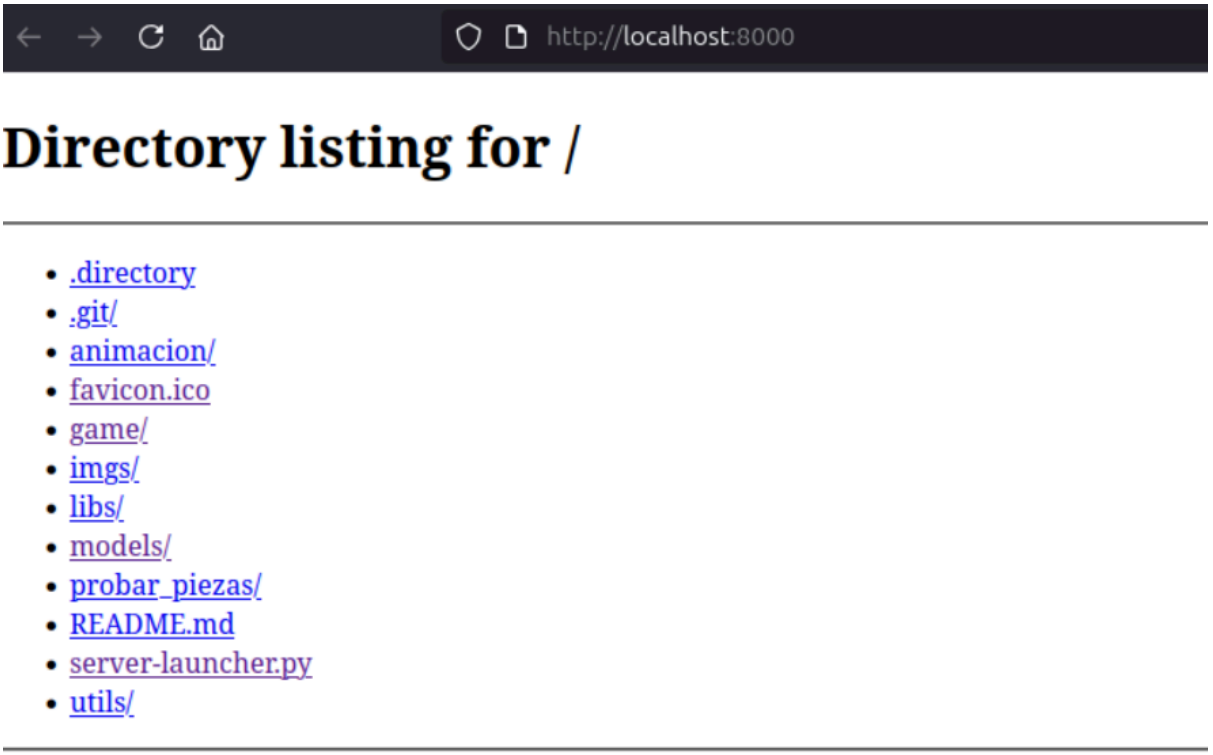
En primer lugar ejecutamos el server, para ello, nos situamos en el directorio raíz del proyecto y ejecutamos:

$ cd codigo/chess_war/
$ python3 server-launcher.py

y nos aparecerá lo siguiente:
Serving at port 8000

application/javascript

Una vez hecho esto, estaremos corriendo el servidor dentro de codigo/chess_war. Bien, ahora buscamos en nuestro navegador la siguiente URL (nosotros lo hemos probado en Chrome y Firefox y no hemos tenido problemas): http://localhost:8000/

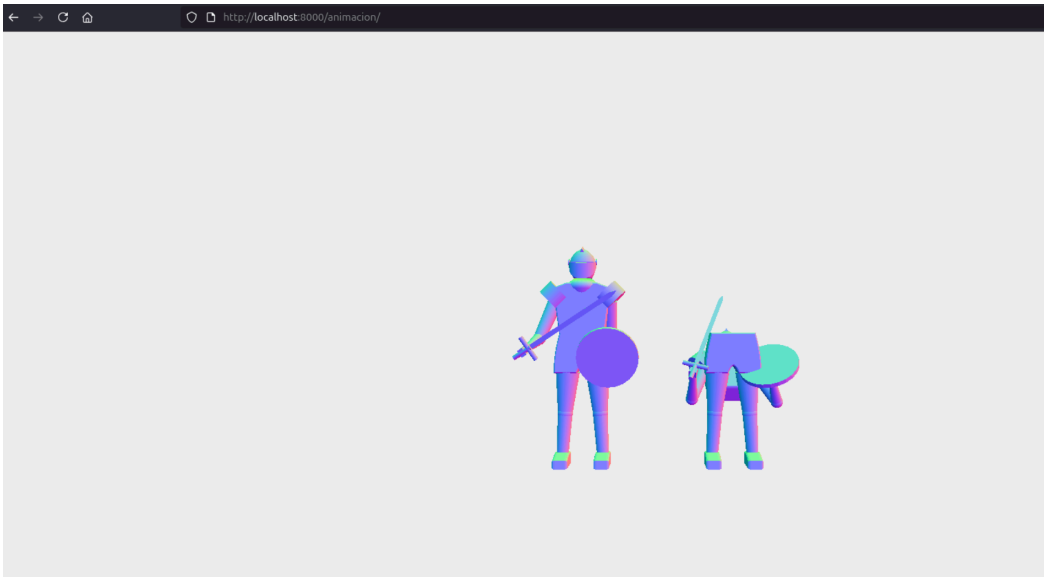


En este punto, encontraremos las carpetas/aplicaciones de codigo/chess/war , las que nos interesan son:

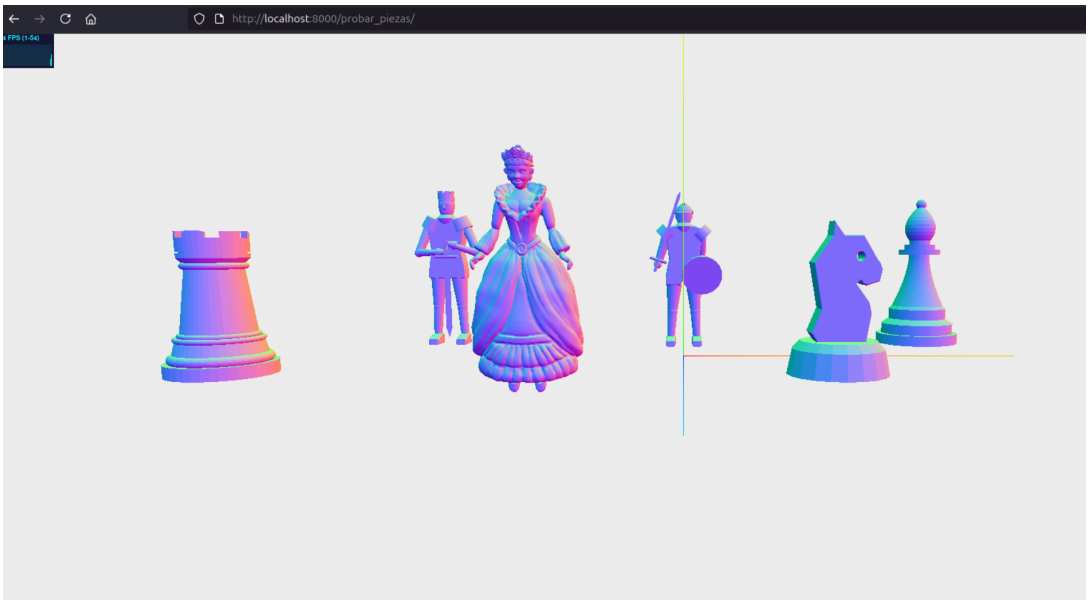
- game/ → Si pinchamos, entraremos al juego. **Es la más importante.**



- animacion/ → Si pinchamos, podremos ver una escena en la que se vean en acción la pieza que lucha para capturar otra pieza. En este caso, el peón que ataca está a la izquierda y el que se defiende es el de la derecha.



- probar_piezas/ →Se muestra con más detalle una escena donde se pueden ver las 6 piezas del ajedrez, el tablero lo hemos omitido, pues son 64 cajas y en el uego se ven perfectamente.



3. Documentación entregada junto al código.

Como podemos observar en la estructura del proyecto, hay un directorio llamado *documentacion/* a la misma altura que *código/*.

En su interior encontramos:

- concrecion_plantilla_chess_war.pdf. Es la concrención sobre la práctica 2 que se entregó al principio. Han cambiado algunas cosas, como la animación del Caballo y del Rey que no la hemos implementado, pero la esencia del juego la hemos mantenido, así como la animación del Peón.

- Diagrama_clases_chess_war.png Es el diagrama de clases de nuestro proyecto. Para simplificar el diagrama, se han resumido el nombre en la declaración de algunos métodos y atributos. Estos son los que llevan un asterisco (*) al final de su nombre, que indica que hay 2 o más atributos y métodos que comienzan igual. Por ejemplo, en las variables de clase geometriaPie y geometriaCabeza, o los métodos crearGeometriaPie() o crearGeometriaCabeza(), o los métodos que implementan estas geometrías: crearPie(material) o crearCabeza(material). Si se desea ver con mayor detalle, mirar el documento de diseño de aplicación. Esta notación también se verá a lo largo de este documento.

- Modelo_jerarquico_peon.png Donde se muestra el modelo jerárquico del Peón. Como única anotación, decir que las variables que se utilizan para las transformaciones, en nuestro código sólo se utilizan para poner la pieza en la posición inicial, que es la que se muestra cuando no se está animando el Peón. Pues, cuando se realiza la animación, se modifica directamente las partes correspondientes.

- video_animacion.txt. Contiene el enlace al video que muestra la animación, alojado en mi Google Drive. El enlace es: https://drive.google.com/file/d/1ET3kfzMweUR3NaHcZ7r9GH6SFE0rp1eI/view?usp=drive_link

Como el diagrama de clases y el modelo jerárquico son muy grandes, es mejor verlos en su formatos de imágenes, por eso no los mostramos aquí.

4. Algoritmos principales

4.1. Lógica

esJudagaPosible(posicion_actual, posicion_siguiente):

Esta función del tablero se encarga de comprobar si una jugada es posible. En esta fase no se comprueba el turno del jugador.

Para llevar a cabo esta tarea, comprobamos que haya una pieza en la posición actual indicada. Si es correcta, solicitamos a la pieza todos sus movimientos posibles desde esa posición. Si alguno de ellos coincide con la posición a la que se intenta mover (solo habría uno), devolvemos la jugada sin aplicarla.

getJugadas(tablero, posicion):

Función de pieza lógica que devuelve las jugadas posibles (mover o mover y comer) desde la posición indicada. Cada pieza implementa su propia versión de este método.

Este método se apoya en las funciones de "MovimientosUtils", las cuales son funciones que ayudan a comprobar si un movimiento es posible en un momento específico. Por ejemplo, el peón sabe que puede comer al paso, pero este delega la responsabilidad de comprobar si esa juaga es posible a las funciones mencionadas. Básicamente, el peón trata de añadir el movimiento de comer al paso desde la posición especificada, y las utilidades comprueban que ese movimiento es posible en el momento actual del tablero y las piezas. Esta es la lista de funciones:

- **anadeMovimientoPosicion**
- **anadeMovimientoCome**
- **anadeMovimientoComeAlPaso**
- **anadeMovimientoEnroque**

anadeJugada(tablero, jugada, come, lista_jugadas):

Independientemente de la función que se haya llamado, si se cumplen los requisitos necesarios para que una jugada se pueda aplicar, todas las funciones anteriores desembocan en esta función.

Esta función se encarga de añadir la jugada a la lista actual de jugadas posibles, hay dos posibilidades:

- Come = false: Se realiza una copia del tablero y se aplica la jugada propuesta. Si dicha jugada no provoca un auto jaque, la jugada se añade a la lista.
- Come = true: La jugada se añade si y sólo si la jugada propuesta come a alguna pieza.

aplicarJugada(jugada):

Aplica la jugada pasada.

Mueve la pieza actual a la casilla destino, si se come a alguna pieza, dicha pieza se elimina, si hay algún movimiento extra (como lo tiene el enroque) se aplica, convierte el peón que llegue a la línea contraria en reina y se comprueban ambos jaques.

4.2. Modelos

actualizar():

Método de “PiezaBaseModelo” y “TableroModelo”, este se encarga de actualizar el estado de ambas clases, ya sea para actualizar su información espacial o el estado de animación (se está animando o no).

Aunque existen diferencias entre las implementaciones de los métodos de actualización en ambas clases, si nos gustaría mencionar la porción del código que se encarga de indicar si la animación está activa, o acaba de terminar:

```
if (this.dentro_animacion) {
    this.animando = true;
    this.termina_animacion = false;
  }
  else {
    if (this.animando) {
      this.termina_animacion = true;
    }
    else {
      this.termina_animacion = false;
    }

    this.animando = false;
  }
```

Esta porción de código es la utilizada para saber en qué momento la animación ha terminado. Esto es útil para cambiar de estado en la escena.

crearModelo(material):

Este método se encarga de llamar a los métodos crear*(). Crea el objeto por completo.

crear*(material):

Se encarga de crear las distintas partes del cuerpo, el asterisco significa que hay varios métodos pero su función es la misma: crearBrazo(), crearCabeza(), etc.

crearGeometria*(material):

Se encarga de crear las geometrías de las distintas partes del cuerpo, el asterisco significa que hay varios métodos pero su función es la misma: crearGeometriaBrazo(), crearGeometriaCabeza(), etc.

animarDefensa()-animarAtaque():

Estos métodos pertenecen a la clase "PeonModelo". Cada método lleva a cabo una animación distinta. Como los nombres ya indican, uno realiza la animación de ataque y otro la de defensa. Ambas animaciones funcionan encadenando animaciones una tras otra hasta llegar a la animación final.

Hay que tener en cuenta que dichas animaciones no duran lo mismo, pero ahí entra en juego el manejo de estados booleanos explicados en la función anterior. La clase que se encarga de asegurarse de que las animaciones hayan terminado antes de pasar al siguiente estado de la aplicación es la escena principal.

4.2. Picking

encontrarObjeto(evento, lista_tipo_piezas, tipo_jugador):

Método de la escena principal.

Este método lanza un rayo desde la posición del ratón que indique el evento, devolviendo el objeto más cercano que lo intersecte y que cumpla con las condiciones especificadas. Se indica el tipo de pieza (peón, caballo, alfil, torre, rey, reina o casilla) y el tipo de jugador (blanco, negro o desconocido, este último para las casillas).

eventoRatonClicArriba(evento):

El método funciona en base a estados:

- **Seleccionando pieza:** Se solicita la pieza que no sea una casilla y que coincida con el jugador actual. La pieza será válida si el número de jugadas posibles es mayor a cero. A continuación, se dibujan en el tablero las jugadas posibles, se ilumina la pieza y pasamos al estado de seleccionar casilla.

- **Seleccionando casilla:** Se ejecuta el lanzador de rayos y se filtra para casillas. Hay dos posibilidades:

    - Si la casilla es la misma donde se encuentra la pieza seleccionada, se apaga la luz de selección, se deselecciona la pieza, se resetea el color del tablero y volvemos al estado de seleccionar pieza.

- Si la casilla es distinta, se trata de aplicar la jugada. Si esto tiene éxito, se anima la jugada.

eventoRatonMovimiento(evento):

Este método ejecuta el mismo código de la sección de seleccionar pieza del método anterior, pero en este caso, no cambiamos de estado ni seleccionamos la pieza. Además de que solo se ejecuta si nos encontramos en el estado de selección de piezas.